

VARIABLES EN CSS (CUSTOM PROPERTIES)

Permiten definir valores una sola vez (como colores o fuentes) y almacenarlos para reutilizarlos en toda la hoja de estilos facilitando el mantenimiento al permitir cambiar un valor desde un solo lugar para actualizar toda la web, manteniendo la coherencia visual y reduciendo la repetición de código.

Son ideales para crear temas dinámicos, como el modo oscuro.

Se declaran globalmente en el selector `:root` usando el prefijo `--` (ej. `--color-principal: #1D3557`) y se aplican a los elementos utilizando la función `var()` (ej. `color: var(--color-principal)`).

Consejo profesional: Organiza tus variables por categorías (colores, tipografía, espaciado) y documéntalas para que otros desarrolladores entiendan su propósito. Esto mejora significativamente la colaboración en equipos.

POSICIONAMIENTO: FIXED

Fija un elemento respecto a la ventana del navegador (viewport), haciendo que permanezca visible en la misma posición aunque el usuario haga *scroll*.

Ideal para menús de navegación, botones de "volver arriba", chats flotantes y avisos de cookies.

Al sacar el elemento del flujo normal del documento, no ocupa espacio y sin un `z-index` adecuado y márgenes en el contenido principal, puede superponerse a otro contenido importante. Además, suele dar problemas de usabilidad en dispositivos móviles, por lo que se recomienda ocultarlos o utilizar `position: sticky` como alternativa.

PROPIEDAD FLOAT (CONCEPTO CLÁSICO)

Originalmente diseñada para hacer que el texto fluyera alrededor de imágenes (como en un periódico), esta propiedad se usó incorrectamente en los años 90 y 2000 para crear estructuras de página completas.

El Problema: Su uso para maquetación es obsoleto y frágil. Los elementos flotados no aportan altura a sus contenedores, causando colapsos que requerían código extra para solucionarlos (el famoso truco *Clearfix*).

Hoy en día, float solo debe usarse para su propósito original: integrar imágenes con texto. **Para crear *layouts* (filas y columnas), se deben utilizar las tecnologías modernas Flexbox y CSS Grid.**

display: flex → para **layouts unidimensionales** (filas o columnas).

display: grid → para **layouts bidimensionales** (filas Y columnas).

position → para posicionamiento específico de elementos.

MEDIA QUERIES (DISEÑO RESPONSIVE)

Son la base del diseño *responsive*, ya que permiten aplicar reglas CSS distintas dependiendo del tamaño de la pantalla del dispositivo. Esto permite adaptar una misma web para móviles, tablets y pantallas de escritorio.

Enfoques:

- **Mobile First (Recomendado):** Se diseña la web primero para móviles y, mediante min-width, se van añadiendo mejoras y complejidad para pantallas más grandes. Es la mejor práctica porque prioriza el contenido esencial y mejora el rendimiento.
- **Desktop First:** Se diseña primero para escritorio y se adapta hacia abajo usando max-width.

Buenas Prácticas:

- Se deben organizar y agrupar al final de la hoja de estilos.
- No deben usarse solo para cambiar colores o fuentes, sino para transformar la estructura del diseño (ej. pasar de 1 columna en móvil a 3 en escritorio).
- No hay que abusar de ellas; el diseño base debe ser fluido usando unidades relativas (% , rem , em), dejando las *media queries* solo como puntos de inflexión.

Breakpoints comunes

Tamaño	Uso típico
480px	Móviles pequeños (iPhone SE, etc.)
768px	Tablets y móviles grandes en horizontal
992px	Portátiles y pantallas medianas
1200px	Escritorio y pantallas grandes
1400px+	Monitores extra grandes y 4K